Aula 8: O Molde Invisível - Classes Abstratas



Projeto: FiapRide + Seu Projeto Pessoal

Foco: Classes Abstratas, Métodos Abstratos e Design de Hierarquias

🎯 *Objetivo da Aula*

Na aula passada, aprendemos sobre Polimorfismo e vimos que a classe `Veiculo` servia como mãe para `Carro` e `Moto`. Mas existe um problema lógico: alguém pode fazer `new Veiculo()` e criar um "veículo genérico" que não é Carro, nem Moto, nem nada!

Pense bem: Você já viu no mundo real um objeto que é APENAS "Veículo"? Um carro é um veículo. Uma moto é um veículo. Mas existe alguma coisa que é só veículo, sem ser nada mais específico?

*Hoje você vai aprender sobre **Classes Abstratas**. Vamos transformar nossa classe `Veiculo` em um molde puro que DEFINE o que é ser um veículo, mas não permite criar instâncias genéricas.*

Conceitos Chave:

- ***Classe Abstrata:** Uma classe que não pode ser instanciada, apenas herdada.*
- ***Método Abstrato:** Um método sem corpo (sem implementação) que OBRIGA as filhas a implementar.*
- ***Molde Puro:** Define a ESTRUTURA que as filhas devem seguir, mas não existe como objeto.*

📖 A Teoria: A Forma do Biscoito vs. O Biscoito

Imagine uma forma (molde) de biscoito em forma de estrela.

- **A Forma (Classe Abstrata):** Existe, você pode tocar nela. Ela define que todos os biscoitos feitos com ela terão formato de estrela. Mas você não pode "comer" a forma. Ela não é um biscoito, é o MOLDE do biscoito.
- **Os Biscoitos (Classes Concretas):** São as instâncias reais. Cada biscoito é uma estrela, mas podem ter detalhes diferentes (chocolate, baunilha, gotas de chocolate).
- **Regra de Ouro:** Se você tentar "assar" a forma do biscoito, não vai funcionar! Você precisa assar os biscoitos DE VERDADE (as subclasses concretas).

Na Programação Orientada a Objetos:

- `Veiculo` (abstrata): Define que todo veículo tem `placa` e `modelo`, mas não existe "veículo genérico" no mundo real.
- `Carro`, `Moto`, `Bicicleta` (concretas): São os biscoitos de verdade. Podem ser criados e usados.

> 📝 **Resumo - A Jornada das Aulas 1-7:**
>
> - **Aulas 1-7:** Trabalhamos APENAS com **classes concretas**. Todas podiam ser instanciadas com `new`.
> - **Aula 8:** Agora aprendemos sobre **classes abstratas**, que NÃO podem ser instanciadas.
> - **Diferença chave:** Classes concretas = podem ser criadas com `new`. Classes abstratas = apenas servem como mães para herança.
> - **No FiapRide:** `Carro` e `Moto` são CONCRETAS (podemos criar objetos). `Veiculo` será ABSTRATA (apenas define estrutura).

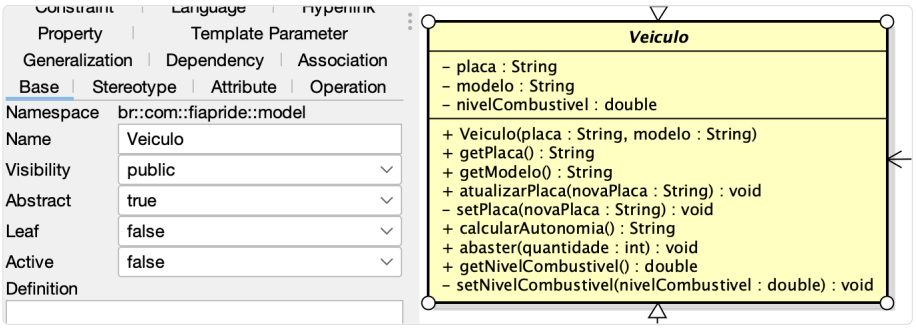
Quando usar Abstract vs. Concrete:

- **Use ABSTRATA se:** A classe representa um conceito genérico que nunca existe sozinho.
- **Use CONCRETA se:** Você precisa criar objetos diretos dessa classe.

🛠️ Parte 1: Modelagem Visual (Astah)

Vamos marcar nossa classe `Veiculo` como abstrata no diagrama.

1. Abra o **Astah** no seu projeto FiapRide.
2. Clique na classe `Veiculo`.
3. Na lateral esquerda selecione a caixinha **Abstract** como **true**.
4. Observe que o nome da classe agora aparece em **itálico** no diagrama!

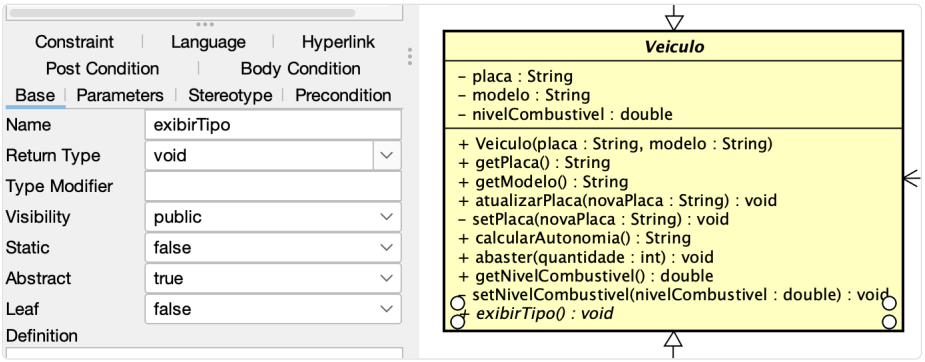


Nota: No UML, classes abstratas são representadas com o nome em itálico. Isso indica visualmente que "você não pode criar objetos dessa classe, apenas herdar dela".

Adicionando Método Abstrato:

Vamos criar um método abstrato que OBRIGA todas as filhas a se definirem.

1. Clique com o direito na classe `Veiculo` > Add > Operation.
2. Nome: `exibirTipo`
3. Deixe o retorno como `void`
4. Na lateral esquerda selecione a caixinha **Abstract** como **true**.



O método aparecerá em itálico, indicando que é abstrato (sem corpo).

💻 Parte 2: O Desafio FiapRide (Mão na Massa)

Transformando Veiculo em Abstract

Vá na classe `Veiculo` e adicione a palavra `abstract`.

```
package br.com.fiapride.model;

// Adicionamos 'abstract' - agora esta classe NÃO pode ser instanciada!
public abstract class Veiculo {

    // ... atributos existentes ...
    private String placa;
    private String modelo;

    // CONSTRUTOR AINDA EXISTE!
    // Classes abstratas TEM construtor, mas só são chamados pelas filhas via super()
    public Veiculo(String placa, String modelo) {
        this.setPlaca(placa);
        this.modelo = modelo;
    }

    // MÉTODO CONCRETO (com corpo)
    // Mesmo sendo classe abstrata, pode ter métodos normais
    public String getModelo() {
        return this.modelo;
    }
```

```
    public String getPlaca() {
        return this.placa;
    }

    // MÉTODO ABSTRATO (sem corpo)
    // OBRIGA todas as filhas a implementarem este método!
    public abstract void exibirTipo();

    // Setters privados...
    private void setPlaca(String placa) {
        if (placa != null && placa.length() == 7) {
            this.placa = placa;
        }
    }
}
```

🧠 **Entendendo o Código:**

- **Linha 4:** `abstract class` - Ninguém pode fazer `new Veiculo()`
- **Linhas 12-15:** Construtor ainda existe (chamado pelas filhas via `super()`)
- **Linhas 27-29:** Método abstrato - Não tem chaves `{}`, termina com `;` direto
- **Obrigatoriedade:** Se `Carro` ou `Moto` não implementarem `exibirTipo()`, o código não compila!

🔍 Por que `abstract public` é REDUNDANTE?

Você pode ter notado que escrevemos:

```
public abstract void exibirTipo();  // ✅ ORDEM CORRETA
```

Mas **NUNCA** escrevemos:

```
abstract public void exibirTipo();  // ❌ ORDEM ERRADA (mas compila!)
```

📝 A Regra de Ouro

> **Métodos abstratos são IMPLICITAMENTE `public`**
>
> **Escrever `abstract public` é redundante, mas FUNCIONA.**

🤔 Por que isso acontece?

Modificador	O que faz	É implícito em `abstract`?
`public`	Pode ser acessado por qualquer classe	✅ **SIM**
`protected`	Apenas filhas acessam	❌ Não
`private`	Apenas a própria classe	❌ **IMPOSSÍVEL** em abstrato
`abstract`	Método sem corpo, obrigatório implementar	-

Explicação:

- Métodos abstratos **EXISTEM para serem sobrescritos** pelas filhas
- Se fosse `private`, ninguém poderia acessar → **contradiz o propósito**
- Se fosse `protected`, apenas filhas do mesmo pacote acessariam → **limita demais**
- Logo, `abstract` **IMPLICA** `public` automaticamente!

📊 Comparação: O que compila vs. o que é correto

Combinação	Compila?	Correto?	Explicação
`public abstract void exibirTipo()`	✅ Sim	✅ **SIM**	Ordem padrão, recomendada
`abstract public void exibirTipo()`	⚠️ Sim	❌ Não	Funciona, mas não é padrão
`protected abstract void exibirTipo()`	✅ Sim	⚠️ Raro	Funciona, mas limita acesso
`private abstract void exibirTipo()`	❌ **NÃO**	❌ Não	**IMPOSSÍVEL!** Contradiz propósito
`abstract void exibirTipo()`	✅ Sim	⚠️ OK	`public` é implícito (fica implícito)

💥 Erro Clássico: `private abstract`

```
// ❌ NÃO COMPILA!
public abstract class Veiculo {
    private abstract void exibirTipo();  // 💥 ERRO!
}

// 🔴 ERRO DE COMPILAÇÃO:
❌ illegal combination of modifiers: abstract and private
```

Por que não compila?

- `abstract` = "precisa ser sobrescrito pela filha"
- `private` = "só esta classe acessa"
- **CONTRADIÇÃO!** Se é privado, a filha não pode sobrescrever!

📊 Classes Concretas vs. Classes Abstratas

> **🎯 A Grande Diferença:**
>
> **Classe Concreta** = Pode ser instanciada com `new`
>
> **Classe Abstrata** = NÃO pode ser instanciada, apenas herdada

Tabela Comparativa Completa

Característica	Classe Concreta	Classe Abstrata
Pode ser instanciada?	✅ SIM (pode usar `new`)	❌ NÃO (erro de compilação)

Característica	Classe Concreta	Classe Abstrata
Pode herdar de outra?	✅ SIM (pode estender outra)	✅ SIM (pode estender outra)
Pode ter métodos concretos?	✅ SIM (com corpo)	✅ SIM (com corpo)
Pode ter métodos abstratos?	❌ NÃO	✅ SIM (sem corpo, obrigatório)
Pode ser filha?	✅ SIM (concreta estendendo concreta/abstrata)	✅ SIM (abstrata estendendo abstrata)
Pode ser mãe?	✅ SIM (filhas concretas herdam dela)	✅ SIM (filhas concretas herdam dela)
Exemplo no FiapRide	`Carro`, `Moto`	`Veiculo`
Palavra-chave Java	(nenhuma)	`abstract`
Quando usar?	Quando precisa criar objetos	Quando é conceito genérico apenas

💡 Regra de Ouro para Identificar

> **Pergunta: "Posso criar um objeto dessa classe no mundo real?"**
>
> - ✅ SIM → É **CLASSE CONCRETA**
> - ❌ NÃO → É **CLASSE ABSTRATA**

Exemplos:

- **Carro:** Posso ter um carro concreto? ✅ SIM → Classe Concreta
- **Moto:** Posso ter uma moto concreta? ✅ SIM → Classe Concreta
- **Veículo:** Posso ter um "veículo" que não é carro, nem moto, nem nada? ❌ NÃO → Classe Abstrata
- **FormaPagamento:** Posso ter uma "forma de pagamento" genérica? ❌ NÃO → Classe Abstrata
- **CartaoCredito:** Posso ter um cartão de crédito? ✅ SIM → Classe Concreta

---

Implementando Método Abstrato em Carro

Agora a filha `Carro` é OBRIGADA a implementar `exibirTipo()`.

```
package br.com.fiapride.model;

public class Carro extends Veiculo {

    private int capacidadePassageiros;

    public Carro(String placa, String modelo) {
        super(placa, modelo);
        this.setCapacidadePassageiros(4);
    }

    // @Override NÃO é opcional aqui!
    // Se não implementarmos, o Java dá erro de compilação
```

```
    @Override
    public void exibirTipo() {
        System.out.println("Sou um Carro");
    }

    @Override
    public String calcularAutonomia() {
        double kmRestante = this.getNivelCombustivel() * 10.0;
        return "Autonomia: " + kmRestante + " km (Consumo de 10 km/l).";
    }

    public int getCapacidadePassageiros() {
        return this.capacidadePassageiros;
    }

    private void setCapacidadePassageiros(int capacidadePassageiros) {
        this.capacidadePassageiros = capacidadePassageiros;
    }
}
```

⚠️ O Que Acontece se ESQUECER de Implementar?

Veja o que ocorre se criarmos uma classe `Bicicleta` e **esquecermos** de implementar `exibirTipo()`:

```
// ❌ ERRADO: Esqueceu de implementar método abstrato
package br.com.fiapride.model;

public class Bicicleta extends Veiculo {

    private int marcha;

    public Bicicleta(String placa, String modelo, int marcha) {
        super(placa, modelo);
        this.marcha = marcha;
    }

    // ❌ ESQUECEU de implementar exibirTipo()!!!
    // O método abstrato da mãe Veiculo não foi implementado
}
```

🔴 Erro de Compilação

```
❌ COMPILER ERROR:
--------------------------------------------------
Bicicleta is not abstract and does not override
abstract method exibirTipo() in Veiculo
--------------------------------------------------
```

📝 Explicação do Erro

Componente do Erro	Significado
`Bicicleta is not abstract`	A classe filha é CONCRETA (não é abstrata)
`does not override`	Não sobrescreveu (não implementou)
`abstract method exibirTipo()`	O método abstrato da mãe
`in Veiculo`	Que está definido na classe Veiculo

🎯 Como Resolver?

Opção 1: Implementar o método ✅ (RECOMENDADO)

```
public class Bicicleta extends Veiculo {

    // ... construtor ...

    @Override
    public void exibirTipo() {  // ✅ IMPLEMENTOU!
        System.out.println("Sou uma Bicicleta");
    }
}
```

Opção 2: Tornar Bicicleta abstrata também ⚠️ (adiar o problema)

```
public abstract class Bicicleta extends Veiculo {
    // ✅ Agora Bicicleta não PRECISA implementar
    // Mas quem HERDAR de Bicicleta PRECISARÁ!
}
```

📊 Comparação: Com vs. Sem Implementação

Situação	Resultado
Esquece de implementar	❌ Não compila
Implementa com @Override	✅ Compila e funciona
Torna filha abstrata	⚠️ Compila, mas adia problema
Erro de digitação no método	❌ @Override previene isso

💡 Regra de Ouro

> **TODO método abstrato PRECISA ser implementado pela filha CONCRETA.**
>
> **Compilador é seu amigo:** Ele não deixa você esquecer!

Implementando Método Abstrato em Moto

```
package br.com.fiapride.model;

public class Moto extends Veiculo {

    private boolean isEletrica;

    public Moto(String placa, String modelo) {
        super(placa, modelo);
    }

    @Override
    public void exibirTipo() {
        System.out.println("Sou uma Moto");
    }

    @Override
    public String calcularAutonomia() {
        double kmRestante = this.getNivelCombustivel() * 35.0;
        return "Autonomia: " + kmRestante + " km (Consumo de 35 km/l).";
    }

    public boolean isEletrica() {
        return this.isEletrica;
    }

    public void setIsEletrica (boolean isEletrica) {
        this.isEletrica = isEletrica;
    }
}
```

O Teste de Compilação (SistemaPrincipal)

Vamos provar que classes abstratas NÃO podem ser instanciadas.

```
package br.com.fiapride.main;

import br.com.fiapride.model.Carro;
import br.com.fiapride.model.Moto;
import br.com.fiapride.model.Veiculo;

public class SistemaPrincipal {
    public static void main(String[] args) {

        System.out.println("=== FIAPRIDE: Teste de Classes Abstratas ===");
        System.out.println();

        // TESTE 1: Tentar criar Veiculo direto (VAI DAR ERRO)
        // Descomente a linha abaixo para ver o erro de compilação!
```

```
        // Veiculo v = new Veiculo("ABC-1234", "Generico");
        // ❌ ERRO: "Veiculo is abstract; cannot be instantiated"
        // O Java bloqueia porque Veiculo é um molde, não um objeto real!

        // TESTE 2: Criar objetos concretos (FUNCIONA)
        Veiculo taxi = new Carro("ABC-1234", "Chevrolet Onix");
        Veiculo mototaxi = new Moto("MOT-9999", "Honda Biz");

        // TESTE 3: Chamar método abstrato implementado
        System.out.println("Identificação dos Veículos:");
        taxi.exibirTipo();      // Saída: Sou um Carro
        mototaxi.exibirTipo();  // Saída: Sou uma Moto

        System.out.println();
        System.out.println("=== Polimorfismo com Classe Abstrata ===");

        // Podemos criar um array de Veiculo (tipo abstrato)
        // mas os OBJETOS são instâncias das filhas concretas
        Veiculo[] frota = new Veiculo[] {
            new Carro("CAR-1111", "Fiat Uno"),
            new Moto("MOT-2222", "Yamaha"),
            new Carro("CAR-3333", "Honda Civic")
        };

        for (Veiculo v : frota) {
            v.exibirTipo();
            System.out.println("---");
        }
    }
}
```

Saída esperada no console:

```
=== FIAPRIDE: Teste de Classes Abstratas ===

Identificação dos Veículos:
Sou um Carro
Sou uma Moto

=== Polimorfismo com Classe Abstrata ===
Sou um Carro
---
Sou uma Moto
---
Sou um Carro
---
```

🧠 Checklist: Clean Code (Classes Abstratas)

- [ ] **Use abstract quando a classe for um conceito genérico:** Se "nunca existe sozinha", deve ser abstrata.
- [ ] **Métodos abstratos para contratos:** Use métodos abstratos para OBRIGAR as filhas a implementarem comportamentos essenciais.
- [ ] **Construtor ainda existe:** Classes abstratas TÊM construtor (são chamados pelas filhas).
- [ ] **Documente o contrato:** Explique em comentários POR QUE determinado método é abstrato.
- [ ] **Não exagere:** Não torne tudo abstrato. Use quando faz sentido LOGICO, não para "parecer inteligente".

⚡ Microdesafio Pessoal: Transforme em Abstrata

Volte para a classe do **seu** projeto pessoal.

Sua Missão:

1. Analise sua classe principal (Ex: `Pet`, `Smartphone`, `Garrafa`).
2. Pense: "Faz sentido existir um objeto APENAS `Pet` sem ser cachorro, gato, etc?"
3. Se a resposta for **NÃO**, transforme a classe em `abstract`.
4. Crie um método abstrato que todas as subclasses devem implementar.
   - *Ex: `emitirSom()` abstrato em Pet, implementado em Cachorro e Gato.*
   - *Ex: `tipoDeConector()` abstrato em Smartphone, retornando "USB-C" em Android e "Lightning" em iPhone.*
5. No `main`, tente criar um objeto da classe abstrata e veja o erro de compilação.
6. Depois crie objetos das subclasses e prove que funciona!

🧠 Pergunta de Reflexão

"Pense no mundo real: Faz sentido existir um objeto que é APENAS 'Veículo' sem ser um tipo específico? Você já entrou em uma concessionária e comprou "um veículo" genérico, sem ser carro, moto, caminhão ou nada disso?

Por que, então, no código, precisamos EXPLICITAMENTE dizer ao Java que `Veiculo` é `abstract`? Por que ele não deduz isso sozinho?

Pense: Se esquecermos de colocar `abstract`, qual o risco que corremos? Alguém pode criar `new Veiculo()` e quebrar a lógica do nosso sistema?"

📤 Finalizando (GitHub)

```
git add .
git commit -m "feat: transforma Veiculo em classe abstrata com metodo exibirTipo"
git push origin main
```

Bons estudos!


FIAP
GRADUAÇÃO
Copyright © 2026 Prof. Ygor Anjos
Todos direitos reservados. Reprodução ou divulgação total ou parcial deste documento é expressamente proibido sem o consentimento formal, por escrito, do Professor (autor).